INTERNSHIP PROJECT REPORT | SYNTECXHUB 2026

PSAWG

Password Strength Analyzer
+ Wordlist Generator

A Cross-Platform Cybersecurity Utility | CLI + Web GUI | Python 3.x

Submitted By	Nitesh Verma
Programme	(BCA) Bachelor of Computer Application
Institution	Banwari Lal Jindal Suiwala College, Tosham / CBLU , Bhiwani
Internship	SYNTECXHUB (Remote Virtual)
Duration	1 Month	March 25, 2026 onwards
Submission	Week 1 Project Submission
Date	May 2026

github.com/CyberNiteshHub/Syntecxhub-Cyber-Security-Internship

For Educational & Authorized Security Testing Only

01 — Project Overview

Executive summary and project details

PSAWG (Password Strength Analyzer + Wordlist Generator) is a cross-platform cybersecurity utility developed as part of the SYNTECXHUB remote virtual internship. The tool addresses two critical needs in modern security testing: objectively evaluating password resilience and generating realistic, targeted wordlists for authorized penetration testing scenarios.

Field	Detail
Project Name	PSAWG — Password Strength Analyzer + Wordlist Generator
Version	1.0
Platform	Cross-platform (Windows + Kali Linux)
Language	Python 3.x
Key Library	zxcvbn (Dropbox password strength estimator)
Interface	CLI (Command Line) + Local Web GUI at http://127.0.0.1:8000/
Output	Strength scores, crack-time estimates, custom wordlists (.txt)
Internship	SYNTECXHUB — Week 1 Project Submission
Repository	github.com/CyberNiteshHub/PSAWG

02 — Problem Statement

Why password security is a critical challenge today

Weak and predictable passwords remain the single largest vector for unauthorized account access worldwide. Security professionals require robust tools to quantify password strength objectively and generate realistic attack wordlists for controlled, authorized testing.

- **Human Bias in Password Creation:** Users consistently choose personally meaningful strings (names, birthdays, pet names) that are trivially guessable by targeted attacks.
- **Inadequate Feedback Mechanisms:** Most password meters provide binary feedback (weak/strong) without actionable guidance or real-world crack-time estimates.
- **Generic Wordlists:** Common wordlists like rockyou.txt are not tailored to target-specific contexts, reducing effectiveness in authorized penetration tests.
- **Cross-platform Tooling Gap:** Many security tools are Linux-only, leaving Windows-based security teams underserved.

03 — System Architecture

Technical design and component overview

PSAWG follows a modular, package-based Python architecture under the psawg namespace. The system is composed of four core subsystems that operate independently or together, ensuring clean separation of concerns and easy extensibility.

Module	Responsibility	Technology
Analyzer Engine	Password entropy scoring, pattern detection, crack-time estimation	zxcvbn library
Wordlist Engine	Keyword mutation, combination generation, configurable output limits	itertools + custom logic
Web GUI	Interactive browser-based dashboard serving both features simultaneously	Flask / local HTTP
CLI Interface	Scriptable subcommands for automation, JSON output, and pipeline integration	argparse / subprocess

04 — Core Features

Detailed breakdown of analyzer and wordlist modules

4.1 — Password Strength Analyzer

The analyzer integrates the zxcvbn library, originally developed by Dropbox, which evaluates passwords using pattern matching, dictionary lookups, and entropy calculations rather than simple character-class rules — providing far more realistic strength assessments.

- **Score 0-4:** Returns a numeric strength rating with granular meaning (Too Weak to Very Strong)
- **Crack Time Estimates:** Provides realistic crack durations for online throttled, online unthrottled, offline slow, and offline fast attack scenarios
- **Actionable Feedback:** Human-readable suggestions and warnings specific to detected patterns (keyboard walks, common words, etc.)
- **JSON Mode:** Machine-readable output flag (--json) for integration with CI/CD pipelines and security dashboards

4.2 — Custom Wordlist Generator

The wordlist generator accepts user-supplied keywords and applies configurable mutations to produce a targeted wordlist tailored to a specific subject, making it far more effective than generic lists for authorized penetration testing.

- **Multi-Keyword Support:** Accepts multiple keywords simultaneously (name, pet, date, nickname, etc.)
- **Year Range Injection:** Automatically appends years within a specified range (e.g., 2018-2026) to every base combination
- **Separator Variations:** Generates combinations with custom separators (_, -, .) for realistic password patterns
- **Mutation Logic:** Case variations, leet-speak substitutions (a->4, e->3, i->1, o->0, s->5), prefix/suffix appending
- **Output Control:** Configurable max-words limit (default 50,000) and custom output filename via --out flag

05 — Installation & Setup

Cross-platform environment configuration guide

Step 1 — Create Virtual Environment

```
python -m venv .venv
```

Step 2 — Activate (Choose Your Platform)

Windows (PowerShell)	Kali Linux / Bash
`.venv\Scripts\Activate.ps1`	
`source .venv/bin/activate`	

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Key Dependencies

Package	Purpose	Source
zxcvbn	Password strength estimation engine	Dropbox Research
Flask	Lightweight web framework for GUI server	Pallets Projects
itertools	Efficient combination/permutation generation	Python Standard Library
argparse	Command-line argument parsing	Python Standard Library
webbrowser	Auto-open GUI in default browser	Python Standard Library

06 — Usage & Commands

Complete CLI and GUI reference

6.1 — Analyze Password Strength

```
python -m psawg analyze --password "P@ssw0rd123"
```

```
# JSON output for scripting and pipeline integration

python -m psawg analyze --password "P@ssw0rd123" --json
```

6.2 — Launch Web GUI

The GUI auto-opens at http://127.0.0.1:8000/ in your default browser:

```
python -m psawg web
```

6.3 — Generate Custom Wordlist

Full command with all flags (use \ instead of ^ on Linux):

```
python -m psawg wordlist ^

  --keyword nitesh --keyword nites --keyword 2003 ^

  --keyword "mydog" ^

  --years 2018:2026 ^

  --separators _ - . ^

  --max-words 50000 ^

  --out wordlist.txt
```

6.4 — CLI Flags Reference

Flag	Command	Description
--password	analyze	Target password string to evaluate
--json	analyze	Output results in JSON format for scripting

Flag	Command	Description
--keyword	wordlist	Base keyword — repeatable for multiple keywords
--years	wordlist	Year range in START:END format (e.g., 2018:2026)
--separators	wordlist	Character separators for combination joining
--max-words	wordlist	Maximum number of output entries (default 50,000)
--out	wordlist	Output filename for the generated wordlist file

07 — Technical Implementation

Code structure, logic flow, and key algorithms

PSAWG is structured as a Python package (python -m psawg) enabling clean separation of concerns and easy extensibility. The entry point dispatches to the appropriate submodule based on the subcommand provided by the user.

7.1 — Package Structure

```
psawg/
├── __main__.py       # Entry point — argument dispatch
├── analyzer.py       # zxcvbn integration, score formatting
├── wordlist.py       # Mutation engine, combination generator
├── web.py            # Flask app, route definitions
├── templates/        # HTML templates for GUI
│   └── index.html
└── requirements.txt  # Dependency manifest
```

7.2 — Analyzer Logic Flow

1. User inputs password via CLI argument or GUI form field
2. zxcvbn(password) returns score, crack times, and feedback dictionary
3. Formatter maps numeric score (0-4) to descriptive label and visual indicator
4. Crack times converted to human-readable strings per attack scenario type
5. Result rendered as formatted console output or JSON blob (--json flag)

7.3 — Wordlist Mutation Strategy

- **Base combinations:** All permutations of provided keywords joined with each separator character
- **Case mutations:** lowercase, UPPERCASE, Title Case, and cAmElCaSe variants generated for each base
- **Leet substitutions:** a->4, e->3, i->1, o->0, s->5 applied selectively across all base combinations
- **Year injection:** Each year in specified range appended and prepended to every base combination
- **Deduplication:** Set-based deduplication applied before write; max-words ceiling enforced on final output

08 — Testing & Results

Validation scenarios and sample outputs

8.1 — Analyzer Test Cases

Password Input	Score	Label	Est. Crack Time (Offline Fast)
password	0 / Too Weak	Too Weak	< 1 second
P@ssw0rd	1 / Weak	Weak	~2 minutes
P@ssw0rd123	2 / Fair	Fair	~3 hours
Tr0ub4dor&3	3 / Strong	Strong	~centuries
correcthorsebatterystaple	4 / Very Strong	Very Strong	~centuries

8.2 — Wordlist Generator Sample Output

```
Keywords      : nitesh, nites, 2003, mydog

Years         : 2018 to 2026  (9 years injected)

Separators    : _  -  .

Mutations     : base, UPPER, Title, leet

_____________________________________________

Total entries generated  : 48,372

Unique after dedup       : 41,889

Output file              : wordlist.txt  (1.2 MB)

Time taken               : 0.84 seconds
```

09 — Ethical Considerations

Responsible disclosure and authorized use policy

!

IMPORTANT — Authorized Use Only

PSAWG is developed exclusively for educational purposes and authorized security testing. Using this tool on systems, accounts, or networks without explicit written permission from the owner is illegal and unethical under the IT Act 2000 (India) and international law.

- **Written Authorization:** Always obtain explicit written permission before testing any system or account you do not personally own.
- **Scope Definition:** Clearly define the scope and boundaries of any penetration test before using wordlist-based approaches.
- **Data Handling:** Generated wordlists may contain sensitive personal information — store securely and delete after authorized testing.
- **Responsible Disclosure:** Any vulnerabilities discovered must be reported responsibly to the affected organization.
- **Legal Compliance:** Comply with IT Act 2000 (India), CFAA (USA), and all relevant jurisdiction-specific regulations.
- **No Malicious Use:** This tool must not be used for credential stuffing, account takeover, or any unauthorized access attempts.

10 — Conclusion

Summary, learning outcomes, and professional takeaways

PSAWG represents a successful synthesis of theoretical cybersecurity knowledge and practical software engineering skills. The project demonstrates proficiency in Python package development, third-party library integration, cross-platform compatibility, and security-conscious design principles — all core competencies for a career in cybersecurity.

- **Technical Skills:** Deepened understanding of password entropy, dictionary attacks, and Python CLI/web application development.
- **Security Awareness:** Hands-on experience with industry-standard tools and real-world attack vectors used in authorized penetration testing.
- **Software Engineering:** Applied modular package design, virtual environment management, and cross-platform compatibility techniques.
- **Documentation:** Developed comprehensive technical documentation covering architecture, usage, and ethical guidelines.
- **Internship Objective:** Successfully fulfilled SYNTECXHUB Week 1 submission requirements, exceeding the basic project scope.

✓ **Project Completed Successfully**

PSAWG v1.0 | Nitesh Verma | BCA 6th Sem, Central University of Haryana | SYNTECXHUB Internship | May 2026

This report was prepared as part of the SYNTECXHUB internship programme. All tools and techniques described are intended for authorized use only.